

Playwright-Automation Training TS/JS :

2026

Writing tests

First test

Playwright tests are simple: they **perform actions** and **assert the state** against expectations. Take a look at the following example to see how to write a test.

```
1  import {chromium, expect, firefox, test, webkit} from '@playwright/test';
2
3  test('Launch URL', async({})=>{
4
5      // const browser = await chromium.launch({channel:'msedge', headless:false});
6      // const browser = await firefox.launch();
7      const browser = await webkit.launch();
8      const context = await browser.newContext();
9      const page = await context.newPage();
10
11     await page.goto("https://google.com");
12     await expect(page).toHaveTitle('Google');
13
14     await page.goto("https://www.facebook.com/");
15     await expect(page).toHaveTitle('https://www.facebook.com/');
16
17 });
```

Working with Locators

Locator is a way to find and interact with elements on a web page. It is built on top of selectors (like CSS or XPath).

These are the recommended built-in locators.

- **page.getByRole()** to locate by explicit and implicit accessibility attributes.
- **page.getByLabel()** to locate a form control by associated label's text.
- **page.getByText()** to locate by text content.
- **page.getByPlaceholder()** to locate an input by placeholder.
- **page.getByAltText()** to locate an element, usually image, by its text alternative.
- **page.getByTitle()** to locate an element by its title attribute.
- **page.getByTestId()** to locate an element based on its data-testid attribute (other attributes can be configured).

```

1  import { expect, test } from '@playwright/test'
2
3  test('Playwright inbuilt locator validation', async ({ page }) => {
4      await page.goto("https://testautomationpractice.blogspot.com/p/playwrightpractice.html");
5      await page.getByRole('textbox', { name: 'Username:' }).fill('Madhan');
6      await page.getByLabel('Email Address:').fill('madhankumar.vanaraj@cognizant.com');
7      await page.getByText('Submit Form').click();
8      await page.getByPlaceholder("Enter your full name").fill('Madhankumar Vanaraj');
9      const playwrightlogo = page.getByAltText('logo image');
10     console.log(playwrightlogo);
11     await page.getByTitle('Home page link').click();
12     const profile = page.getByTestId('edit-profile-btn');
13     await expect(profile).toBeEnabled();
14 });

```

XPath:

XPath (XML Path Language) is a powerful query language used in automation testing to navigate and locate elements in XML or HTML documents.

- **Absolute Xpath:** Absolute XPath provides the full path from the root (html) element to the target element.
- **Relative Xpath:** Relative XPath starts from anywhere in the document and locates elements without relying on the full path.
- **Collection Xpath:** XPath pattern based on the indexes with(out) relations.
- **Parent-Child:** XPath pattern based on the (grand) parent to (grand)child relationships
- **AXES Xpath:** XPath pattern based on the sibling & Cousin relationships.

RELATIVE XPath pattern based on the attribute / text ATTRIBUTE //tagName[@attribute='value'] TEXT //tagName[text()='value'] PARTIAL ATTRIBUTE //tagName[contains(@attribute,'value')] PARTIAL TEXT //tagName[contains(text(),'value')]	PARENT - CHILD XPath pattern based on the (grand) parent to (grand) child relationships PARENT TO CHILD (xpath of parent)/child-tagName CHILD TO PARENT (xpath of child)/parent::parent-tagName CHILD TO GRAND PARENT (xpath of child)/ancestor::ancestor-tagName GRAND PARENT TO CHILD (xpath of ancestor)//child-tagName LAST CHILD (xpath of parent)/child-tagName[last()]
COLLECTION XPath pattern based on the indexes with(out) relations Note: index starts at 1 USING ITEM INDEX (xpath)[index] USING CHILD ITEM INDEX (parent-xpath)/child-tagName[index] USING AND CONDITION //tagName[@att1='v1' and @att2='v2'] USING OR CONDITION //tagName[@att1='v1' or @att2='v2']	AXES XPath pattern based on the sibling & Cousin relationships YOUNGER SIBLING (xpath)/following-sibling::sibling-tagName ELDER SIBLING (xpath)/preceding-sibling::sibling-tagName YOUNGER COUSIN (xpath)/following::cousin-tagName ELDER COUSIN (xpath)/preceding::cousin-tagName

XPATH CHEATSHEET

CSS Selectors:

A CSS Selector is a locator strategy used in automation tools (Selenium, Playwright, Cypress, etc.) to uniquely identify and interact with web elements.

Selector Type	Purpose	Examples
Basic (Simple) Selectors	Select elements by name, id, or class	<code>p</code> , <code>#id</code> , <code>.class</code> , <code>*</code>
Combinator Selectors	Select elements based on relationships (descendant, child, sibling)	<code>div p</code> , <code>div > p</code> , <code>div + p</code> , <code>div ~ p</code>
Attribute Selectors	Select elements based on attributes or attribute values	<code>[type='text']</code> , <code>[href*='example']</code>
Pseudo-class Selectors	Select elements in a specific state	<code>:hover</code> , <code>:focus</code> , <code>:nth-child(2)</code>
Pseudo-element Selectors	Select and style part of an element	<code>::before</code> , <code>::after</code> , <code>::first-line</code>

Thank you

Madhankumar Vanaraj